

廈門大學

中 期 汇 报

# OpenHarmony 向标联合查询中期汇报

汇报人：林正刚

## 年度课题-挑战方向3：向标联合查询性能优化



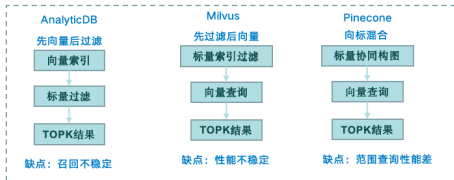
## 技术背景

端侧向量数据库（比如Ark Data自研数据库）通过标量向量数据的统一管理和联合查询，支撑：小艺记忆语义内容查询、小艺搜索本地数字资产语义问答。  
为了在大数据集上快速查询，建立向量索引与标量索引，在联合查询场景下维护两套索引，若选择不好的执行计划，查询时延会大幅上升，对DB带来较大压力。

## 技术挑战

1. 端侧数据库考虑资源功耗问题没有统计信息，无法通过代价计算选择生成执行计划。
2. 构建轻量级混合索引，低底噪、低功耗、低时延查询。

## 当前结果



## 技术诉求

## 实现低功耗稳定低时延的向量标量联合查询。

1. 客观效率方面：百万128维向量与标量数据集下标量条件等值或者范围查询联合向量近似查询中，标量条件命中数量多少不影响时延召回率，10@10召回率>98%；
2. 主观感受方面：在对表中数据查询，对查询条件包含标量等值或这个范围查询并包含向量近似查询召回率与性能稳定，不会由于数据分布的问题导致部分查询出现劣化。

## 分解目标

**动态增量混合索引：**端侧数据是会不断增加更新，索引应可以从0数据量开始构建，增量更新数据不影响召回率。

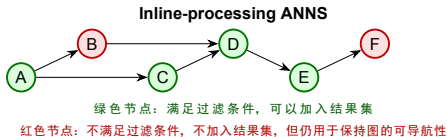
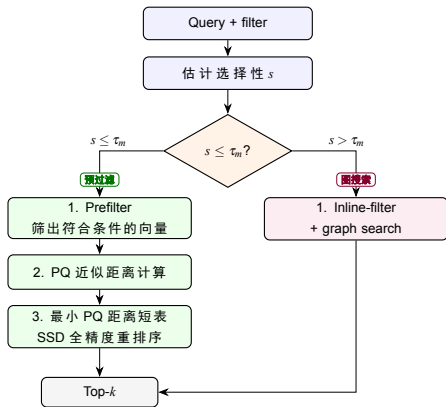
**情景举例：**在端侧用户创建表和索引构建后用户插入大量数据，查询数据满足标量条件以及最近似的查询向量top10，低延时稳定查出结果。

**考量点：**(1)数据不断增删改，查询10@10召回率稳定>98%；(2)索引能够支持标量等值与范围查询，联合向量近似查询时延<10ms；(3)页缓存内存限制10M左右；(4)索引膨胀率不能高于原始向量1倍。(5)基于mate60进行验证。

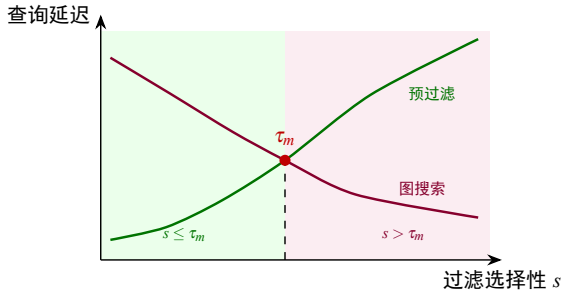
OpenAtom OpenHarmony

| 需求点                 | 状态   | 备注                                                |
|---------------------|------|---------------------------------------------------|
| 动态增量混合索引            | 完成   | 10k 以内走纯内存精确近邻搜索，超过阈值后落盘为磁盘图。                     |
| 查询 10@10 召回率稳定 >98% | 完成   |                                                   |
| 标量等值与范围查询           | 完成   |                                                   |
| 联合向量近似查询时延 <10ms    | 完成   |                                                   |
| 内存限制 30M 左右         | 完成   |                                                   |
| 索引膨胀率不能高于原始向量 1 倍   | 部分完成 | 目前 R=116 情况下索引膨胀率略超；也可以使用 R=75，但插入删除会对图质量有比较大的影响。 |

根据过滤选择性阈值  $\tau_m$ ，在同一个磁盘图索引上选择 prefilter 或 inline-processing ANNS。



标定曲线示意



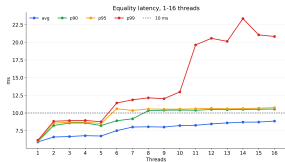
## 曲线含义

- 先过滤：选择性越高，候选越多，延迟上升。
- 图搜索：选择性越高，有效点越密，搜索范围缩小。
- $\tau_m$ ：两条曲线附近的切触点。

## 标定方式

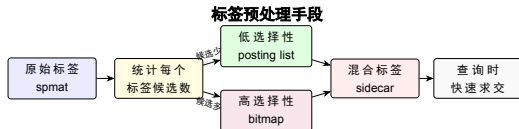
- 采样选择性点，测 prefilter / graph 在 recall@10  $\geq$  98 下的延迟。
- 取两路延迟相近处为  $\tau_m$ ，写入索引元数据。
- 前台搜索线程数低于 16 才启动标定查询；标定期间暂停插入/删除。

1-16 线程搜索延迟



## 16 线程依据

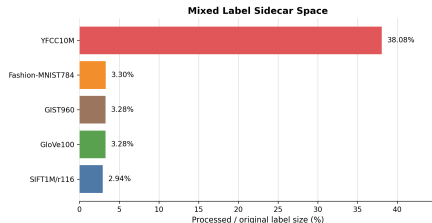
- r116 的 1-16 线程实验中，16 线程全选择性最大 avg latency 为 8.90ms，未超过 10ms。
- 因此采用“前台搜索线程数低于 16 才启动标定”的界线。



按标签选择性选择表示：小集合用 posting list 省空间，大集合用 bitmap 保持稳定求交速度。

## 标签表示

- spmat: 行是向量、列是标签，非零表示持有该标签。
- posting list: `std::vector<uint32_t>`，保存向量 id。
- bitmap: `std::vector<uint64_t>`，每个 bit 表示一个向量。
- 每个标签持有一个 `DenseBitsetLabelEntryV2`，记录 encoding、候选数和 payload 位置。



## 实验结果

- 空间图测量 base spmat 到 mixed sidecar；同一标签文件可被等值 / 范围 selector 共用。
- 5 个数据集 processed/original label size: 2.94%–38.08%。
- 等值 / 范围全选择性:  $\text{recall}@10 \geq 98$ , avg latency  $\leq 8.60\text{ms}$ 。

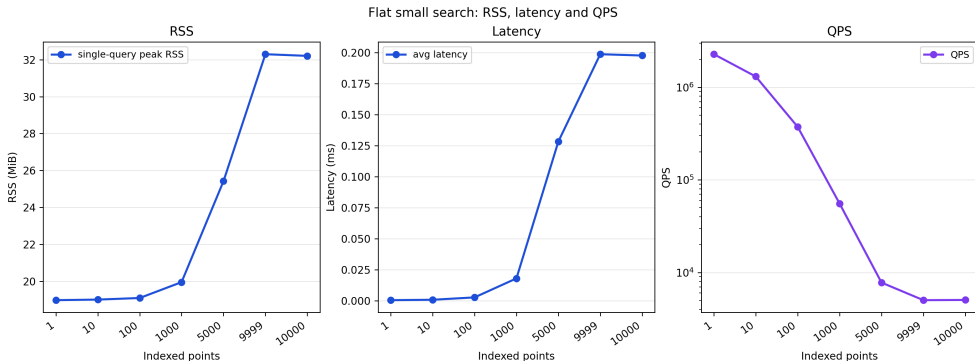
结论：混合表示没有造成标签空间膨胀，并保持 10ms 平均延迟目标。

# 需求 0：从 0 插入与阈值前搜索

从 0 数据量向索引中插入向量时，进入纯内存模式；0–10k 向量直接插入内存数组，并执行精确近邻搜索。该阶段不训练 PQ、不生成磁盘图；索引中向量数超过 10k 后，再一次性落盘为磁盘图索引。

SIFT1M 前 10k 向量实验中，10k 以内全部保持纯内存模式；单查询峰值 RSS 最高约 **32.3 MiB**，接近 30MB 要求；平均搜索延迟最高约 **0.20 ms**，显著低于 10ms 延迟要求，10k 附近 QPS 约 **5k**。

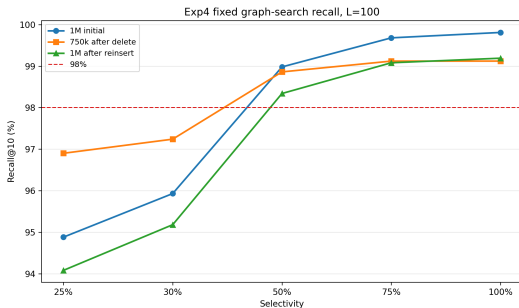
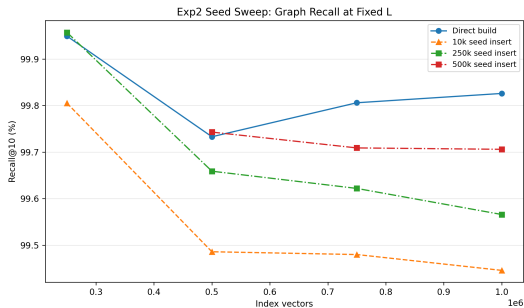
结论：已支持从 0 数据量开始插入；落盘前延迟满足要求，内存占用与 30MB 目标基本一致。



**左图：直接构建 vs 增量插入。**分别直接构建 250k、500k、750k、1M 索引，并与从不同初始规模索引插入到相同规模后的索引比较；两类索引在同一个  $L$  下执行搜索，用于衡量直接构建和增量插入得到的图质量差距。实验中召回率差距在 0.5% 以内，基本没有影响。

**右图：删除 / 重插入后的图质量。**先构建 1M 索引，删除 250k 向量得到 750k 索引，再插入 250k 向量回到 1M，对三个索引状态分别运行搜索测试。结果显示选择性超过 50% 后 recall 差异缩小到 1% 以内。

**结论：删除和插入操作对图质量影响较小。**

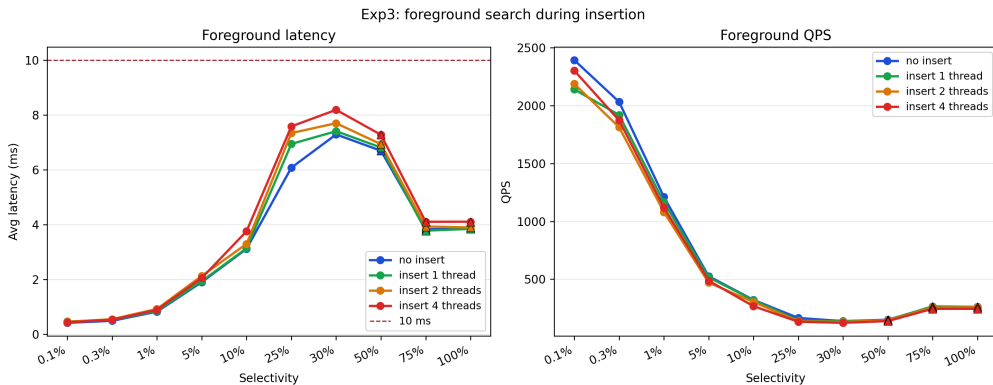




## 需求 2：插入期间前台查询

在初始 1M 索引上插入向量的同时执行前台搜索，测量搜索延迟与 QPS，结果如下图。

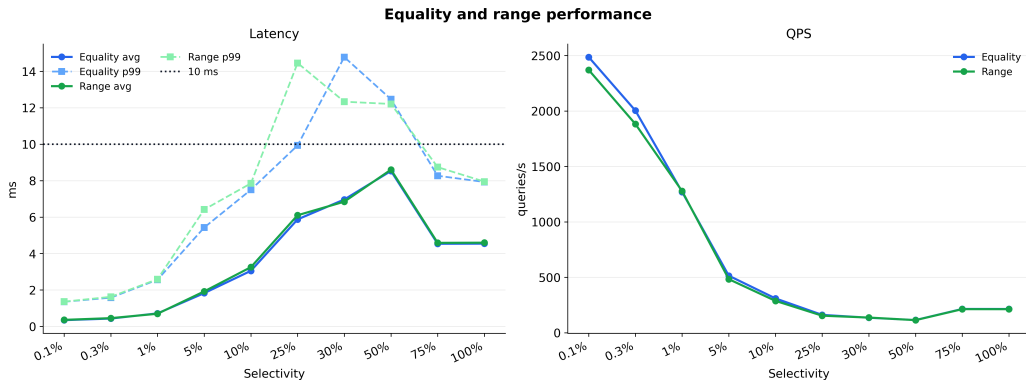
**结论：在插入线程数小于 4 时，向量插入并不会对前台搜索性能产生显著影响。**

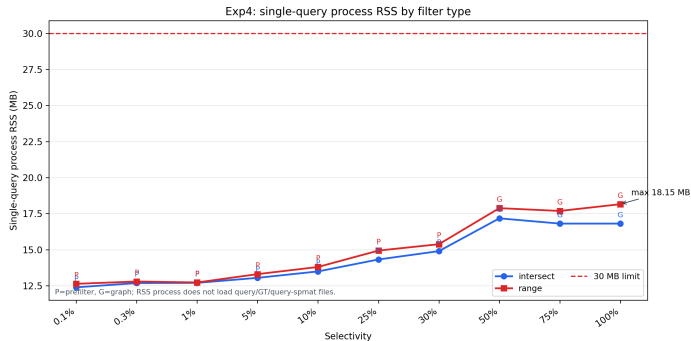


# 需求 3：选择性查询延迟/QPS

1M 索引；等值 / 范围查询；所有点  $\text{recall}@10 \geq 98$ 。

结论：全部选择性 avg latency < 10ms。





## 测量方法

只加载必要文件到内存，完成单个查询的搜索流程并返回结果；不加载 query file、groundtruth 等其他文件。

## 实测结果

20 个点范围为 **12.39–18.15 MiB**；  
等值查询峰值 17.17 MiB，范围查询峰值 18.15 MiB。

结论：满足 30MB 内存要求。

R=116 下，额外索引文件主要来自磁盘图；1M 数据规模下额外空间约为原始向量的 2.06 倍。

| 规模   | 原始向量   | 磁盘图    | PQ 压缩向量 | PQ 码本 | 标签位图 | 额外合计    | 额外/原始 |
|------|--------|--------|---------|-------|------|---------|-------|
| 250k | 122.07 | 244.14 | 7.63    | 0.13  | 0.30 | 252.20  | 2.07x |
| 500k | 244.14 | 488.29 | 15.26   | 0.13  | 0.60 | 504.27  | 2.07x |
| 750k | 366.21 | 732.43 | 22.89   | 0.13  | 0.89 | 756.34  | 2.07x |
| 1M   | 488.28 | 976.57 | 30.52   | 0.13  | 1.19 | 1008.41 | 2.06x |

单位：MiB；额外/原始为额外合计与原始向量的比值。

THE END

谢谢

